

Name: P Neeharika

Roll No: 2520090233

PRACTICAL (OSSP)

SESSION 4

TASK 1: Write a C program where a parent process creates multiple child processes and synchronizes their completion using wait() and waitpid(). Compare the behavior of both functions.

```
GNU nano 7.2 session4_wait.c
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

int main() {
    pid_t children[3];

    printf("Parent process started. PID = %d\n", getpid());

    // Create 3 child processes
    for (int i = 0; i < 3; i++) {
        children[i] = fork();

        if (children[i] < 0) {
            perror("fork failed");
            exit(1);
        }

        if (children[i] == 0) {
            printf("Child %d started. PID = %d\n", i + 1, getpid());
            sleep(2 + i);
            printf("Child %d completed. PID = %d\n", i + 1, getpid());
            exit(0);
        }
    }

    // Using wait() - waits for any child
    printf("\nParent using wait()...\n");

    pid_t finished = wait(NULL);
    printf("wait() collected child PID = %d\n", finished);

    // Using waitpid() - waits for a specific child
    printf("\nParent using waitpid() for child PID = %d...\n", children[1]);

    int status;
    waitpid(children[1], &status, 0);

    printf("waitpid() collected child PID = %d\n", children[1]);

    // Collect remaining children
    for (int i = 0; i < 3; i++) {
        if (children[i] != finished && children[i] != children[1]) {
            waitpid(children[i], NULL, 0);
        }
    }

    printf("\nAll child processes completed.\n");

    return 0;
}
```

^G Help	^O Write Out	^W Where Is	^K Cut	^T Execute	^C Location	M-U Undo
^X Exit	^R Read File	^N Replace	^U Paste	^J Justify	^_ Go To Line	M-E Redo

```
root@HP: ~/shellforge
cd ~/shellforgeroot@HP:~# cd ~/shellforge
root@HP:~/shellforge# nano session4_wait.c
root@HP:~/shellforge# gcc session4_wait.c -o session4_wait
root@HP:~/shellforge# ./session4_wait
Parent process started. PID = 549
Child 1 started. PID = 550
Child 2 started. PID = 551

Parent using wait()...
Child 3 started. PID = 552
Child 1 completed. PID = 550
wait() collected child PID = 550

Parent using waitpid() for child PID = 551...
Child 2 completed. PID = 551
waitpid() collected child PID = 551
Child 3 completed. PID = 552

All child processes completed.
root@HP:~/shellforge#
```

```
GNU nano 7.2 session4_zombie.c
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

int main() {
    pid_t pid = fork();

    if (pid < 0) {
        perror("fork failed");
        exit(1);
    }

    if (pid == 0) {
        printf("Child process running. PID = %d\n", getpid());
        sleep(2);
        printf("Child process exiting...\n");
        exit(0);
    } else {
        printf("Parent process running. PID = %d\n", getpid());
        printf("Child PID = %d\n", pid);

        // Proper synchronization
        waitpid(pid, NULL, 0);

        printf("Parent collected the child using waitpid().\n");
        printf("No zombie process remains.\n");
    }

    return 0;
}
```

TASK 2: Create a scenario where a child process becomes a zombie process.

Investigate the process table and then modify the program to eliminate zombie

processes using proper synchronization techniques.

```
root@HP:~/shellforge# nano session4_zombie.c
root@HP:~/shellforge# gcc session4_zombie.c -o session4_zombie
root@HP:~/shellforge# ./session4_zombie &
[1] 561
root@HP:~/shellforge# Parent process running. PID = 561
Child process running. PID = 562
Child PID = 562
Child process exiting...
Parent sleeping without wait().
ps -l
F S      UID      PID      PPID  C PRI  NI ADDR SZ WCHAN  TTY          TIME CMD
4 S      0        359      358  0  80   0  -  1520 do_wai pts/0        00:00:00 bash
0 S      0        561      359  0  80   0  -   672 hrtime pts/0        00:00:00 sessio
1 Z      0        562      561  0  80   0  -    0 -      pts/0        00:00:00 sessio
4 R      0        563      359  0  80   0  -  2080 -      pts/0        00:00:00 ps
root@HP:~/shellforge# Parent exiting.
|
root@HP:~/shellforge# Parent exiting.
nano session4_zombie.c
[1]+  Done                  ./session4_zombie
root@HP:~/shellforge# gcc session4_zombie.c -o session4_zombie
root@HP:~/shellforge# ./session4_zombie
Parent process running. PID = 570
Child process running. PID = 571
Child PID = 571
Child process exiting...
Parent collected the child using waitpid().
No zombie process remains.
root@HP:~/shellforge# |
root@HP:~/shellforge# ps -o pid,ppid,state,cmd
  PID    PPID S  CMD
   359    358 S  -bash
   575    359 R  ps -o pid,ppid,state,cmd
root@HP:~/shellforge# |
```

Overall Observation

The practical successfully demonstrated **process creation and synchronization in C using `fork()`, `wait()`, and `waitpid()`**. Multiple child processes were created and their completion was synchronized by the parent process. The practical also demonstrated how a child process can become a **zombie process** when the parent does not collect its termination status. By using `waitpid()` for proper synchronization, the parent successfully collected the child process and eliminated the zombie state.

